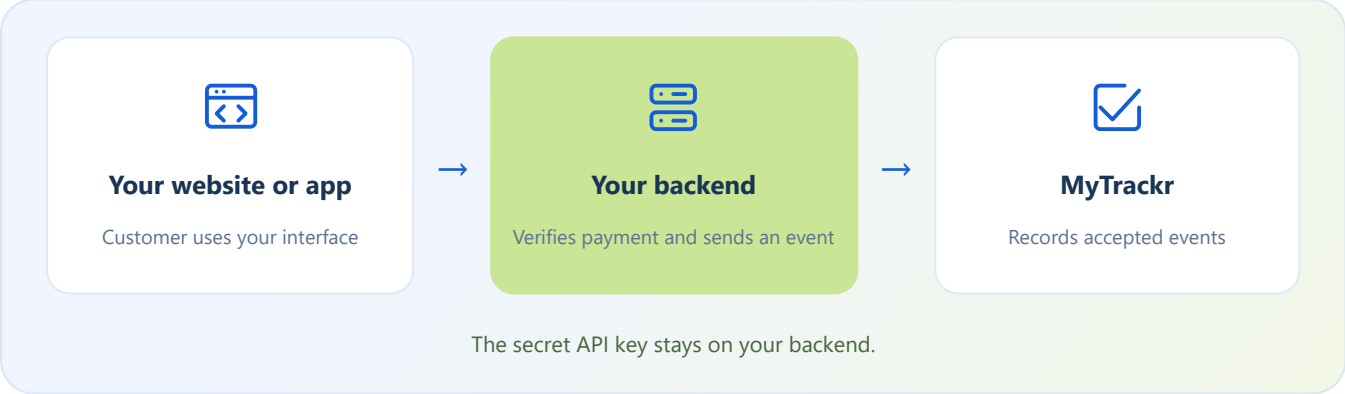
BUILD WITH MYTRACKR

Your website. Your orders. One clear connection.

A visual guide to connecting React, Vue, Flutter, and WooCommerce to MyTrackr.



Open developer dashboard ↗ Explore all online guides ↗

Your route through this handbook

02–03 **Create an integration** — Find and save your credentials

04 **Connect your backend** — Send a verified order event

05–07 **Choose your platform** — React · Vue · Flutter · WordPress

08 **Connect payments** — Find keys and verify with Paystack

09 **Receive webhooks** — Check signatures and prevent repeats

10 **Launch with confidence** — Checklist and troubleshooting

November 2026 · A4 edition · Click the links for complete code examples.
Use a verified MyTrackr account, a plan with website linking, and a backend you control.
Main dashboard screenshots and API samples: pages 11–13.

01 / GET CONNECTED

Create your integration.

Register the website or app you want to connect. One integration groups its credentials and events.

1 Sign in

Open the developer dashboard with your verified account. Google-only accounts first need a password set through account recovery.

2 Check your plan

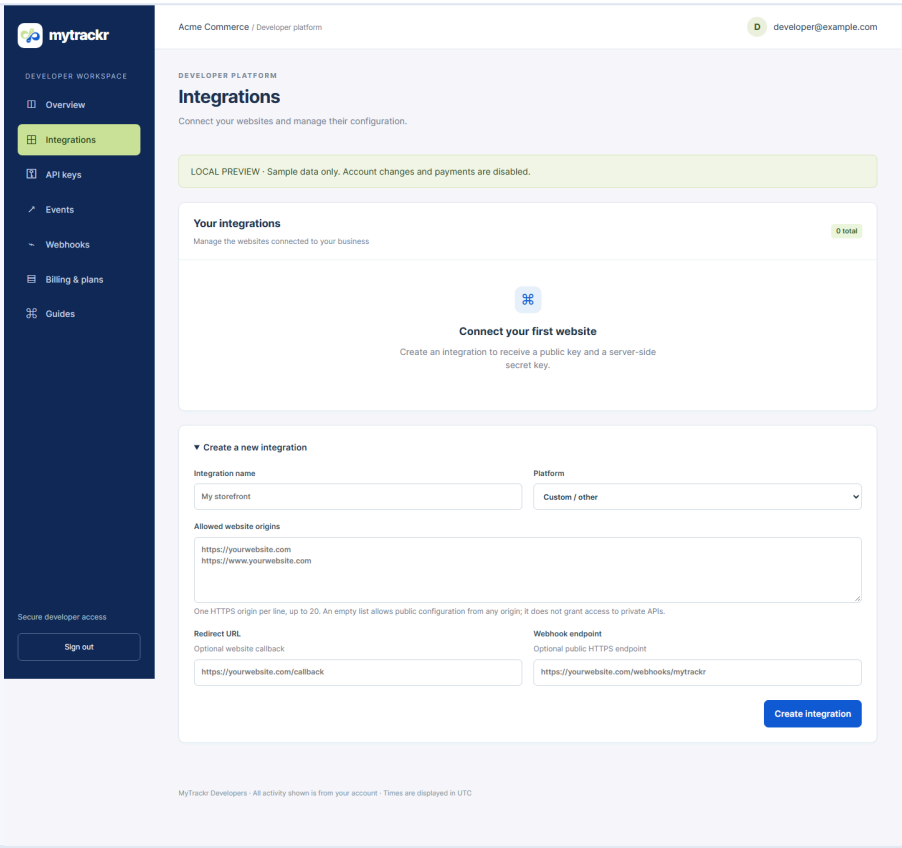
Open **Billing & plans**. Your active plan must include website linking.

3 Choose a platform

Select **Integrations → Create a new integration**. Pick React, WordPress, or Custom / other for Vue and Flutter.

4 Add exact origins

Use `https://shop.example.com`, one per line. Add subdomains separately; omit paths such as `/checkout`.



Snapshot 1 · Where to create a website integration

Actual MyTrackr preview · sample data, not a customer account

Save the response

Creation displays the secret API key and webhook signing secret once. Save both before leaving. Leave the webhook URL blank until your receiver is ready.

02 / FIND YOUR CREDENTIALS

Three keys. Three jobs.

Open API keys and choose your integration. Public credentials and private credentials have different roles.

mt_pk_...

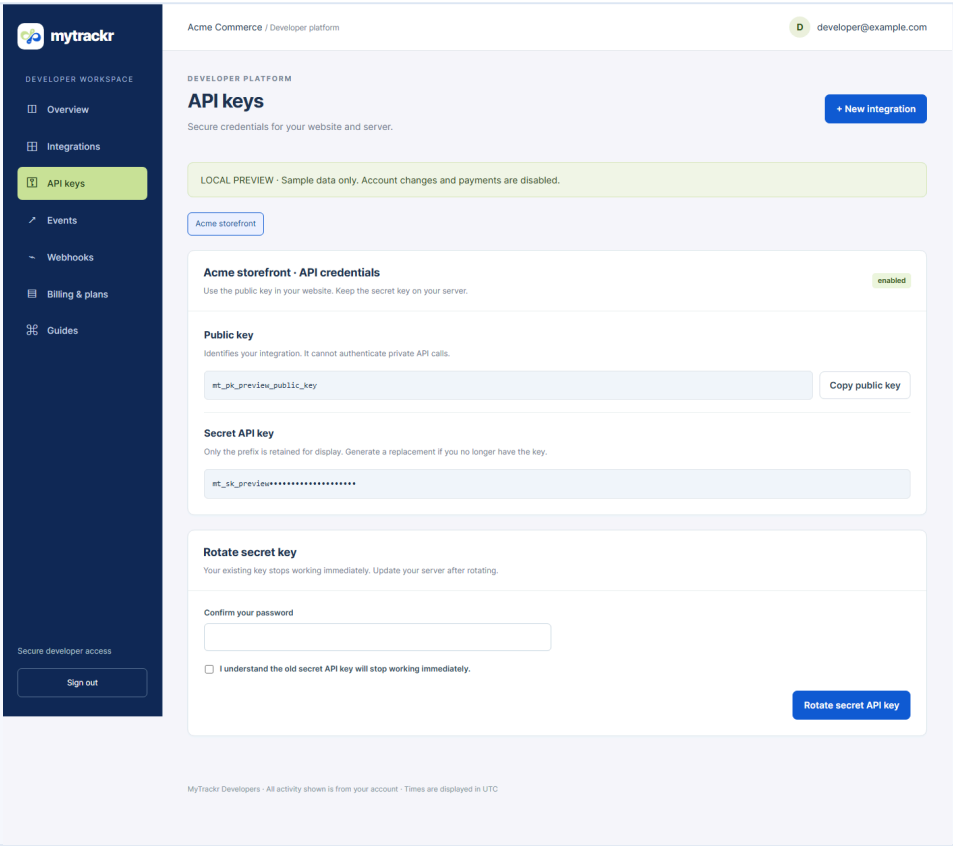
Public integration key. Identifies the integration for public configuration. It cannot authenticate private requests.

mt_sk_...

Secret API key. Your backend sends this in the `x-mytrackr-api-key` header.

mt_whsec_...

Webhook signing secret. Your receiving server uses it to verify notifications from MyTrackr.



Snapshot 2 · Where to find your API keys

Actual MyTrackr preview · sample data, not a customer account

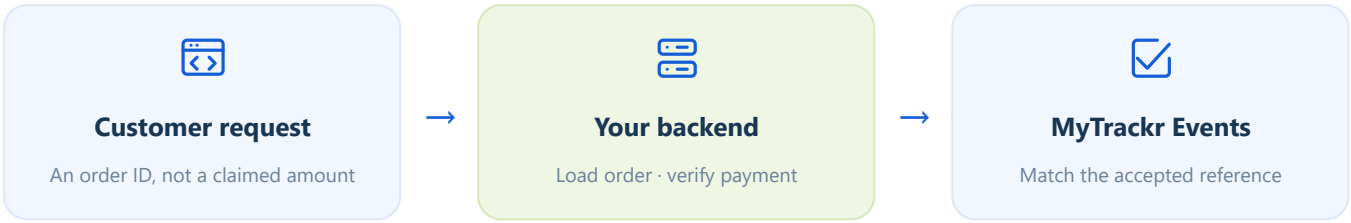
Lost a secret? Rotate it.

The API keys page only shows the secret prefix. Rotate the API key there, or the signing secret under Webhooks, confirm your password, and update your server immediately.

03 / YOUR BACKEND

Send a verified order.

Your server is the trusted bridge. It checks the customer and payment before it records an order in MyTrackr.



1 Keep credentials server-side

Set `MYTRACKR_API_URL` to `https://api.mytrackrservices.com` and `MYTRACKR_API_KEY` to your `mt_sk_…` key.

2 Trust your own order database

Authenticate the customer, check ownership, and verify the payment reference, amount, currency, merchant, and order.

3 Send from the verified payment handler

Prefer a server-side handler or background job. The client sync button is an optional retry for a known paid order.

POST

`/integrations/events`

On the MyTrackr API origin.

Authentication header

```
x-mytrackr-api-key: mt_sk_…
```

Never send the shared secret from a browser or mobile app.

Amount unit

2500 = NGN 2,500

Convert Paystack kobo to naira once.

JSON REQUEST · ILLUSTRATIVE ORDER

```
{
  "event": "order.paid",
  "externalId": "store:1048:paid",
  "orderId": "1048",
  "amount": 2500,
  "currency": "NGN",
  "occurredAt": "2026-11-11T12:00:00Z"
}
```

Retry the same event ID

Keep `externalId` stable after a timeout. A repeated event is acknowledged with `duplicate: true`. Use a different ID for a refund or another event type.

04 / REACT + VUE

A familiar interface. A secure backend.

Both platforms call your application backend. Your backend holds the merchant key and talks to MyTrackr.

React

Choose **React** in Integrations. Add your deployed website's exact HTTPS origins. Use the guide's `SyncOrder.jsx` component.

Vue

Choose **Custom / other**. Add your website origins. Use the guide's `SyncOrder.vue` component; no MyTrackr Vue package is required.

01 **Customer signs in**
Your application session

→

02 **Interface requests sync**
Order ID + your CSRF token

→

03 **Backend validates**
Ownership + verified payment

SHARED BROWSER REQUEST · REQUEST EXCERPT

```
const response = await fetch(
  '/api/orders/' + encodeURIComponent(orderId) + '/sync',
  {
    method: 'POST',
    credentials: 'same-origin',
    headers: { 'X-CSRF-Token': csrfToken },
    signal: AbortSignal.timeout(15000)
  }
);
const result = await response.json();
if (!response.ok || !result.synced) {
  throw new Error('Sync could not be confirmed');
}
```

This route belongs to your app

Implement `POST /api/orders/:orderId/sync` on your backend using the server guide. It is not a built-in MyTrackr endpoint. Adapt CORS and authentication if your backend is on another origin.

- ✓ Disable the sync button while a request is running.
- ✓ On a temporary failure, retry the same order and event ID.
- ✓ Show success only after your backend confirms acceptance.
- ✓ Open Dashboard → Events and match the order reference.

05 / FLUTTER

Connect the app. Keep the secret on the server.

Use the customer's own access token to call your backend. A merchant API key must never ship in the app bundle.

1 Register the integration

Choose **Custom / other**. Native mobile apps have no browser origin; configure origins for Flutter web.

2 Add the HTTP client

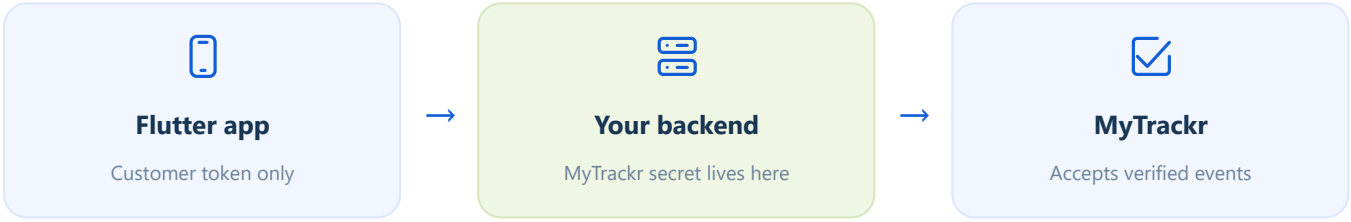
Run `flutter pub add http`. Your app needs its own customer sign-in flow and internet access.

3 Implement your backend route

Use the shared server guide to build `/api/orders/:orderId/sync`. Validate ownership and payment.

4 Connect the order screen

Call the service after customer action, disable repeat taps while busy, and show errors with a retry option.



DART · REQUEST EXCERPT, INSIDE YOUR APP SERVICE

```dart
final uri = Uri.https(
  'your-backend.example.com',
  '/api/orders/' + Uri.encodeComponent(orderId) + '/sync',
);
final response = await http.post(
  uri,
  headers: {'Authorization': 'Bearer ' + customerToken},
).timeout(const Duration(seconds: 15));

// Check the HTTP status and decoded JSON in the full example.
// Show success only when your backend returns synced: true.
```

Widget lifecycle

After an `await`, check `mounted` before updating a widget. Do not leave the button stuck in its busy state.

Release builds still need a backend

Build variables, obfuscation, and device storage do not make a shared merchant secret safe to distribute.

06 / WORDPRESS + WOOCOMMERCE

Connect paid store orders.

Use a custom server-side plugin and WooCommerce's payment-complete flow. Your hosting environment keeps the credentials private.

WordPress → WooCommerce → Payment complete → MyTrackr event

1 Prepare the store

You need hosting or administrator access and WooCommerce installed. Create a MyTrackr integration with **WordPress** selected.

2 Set private configuration

Back up `wp-config.php`. Add the constants below before the "stop editing" line using your host's secure deployment process.

WP-CONFIG.PHP · SERVER CONFIGURATION

```
define('MYTRACKR_API_URL',
  'https://api.mytrackrservices.com');
define('MYTRACKR_API_KEY',
  'mt_sk_REPLACE_WITH_YOUR_SECRET');
```

3 Install the guide's custom plugin

Create `wp-content/plugins/mytrackr-orders/mytrackr-orders.php` and use the complete plugin example in the online guide. Activate **MyTrackr Order Connection** under WordPress → Plugins.



Paid orders

The example sends new payment-complete events. Verify the amount, currency, and order reference in MyTrackr Events.



Scheduled retries

The example retries temporary failures three times through WordPress scheduled events. Your host must run WP-Cron reliably.



Expand deliberately

Historical order imports and refund events are not automatic. Add separate verified handling with distinct event IDs if needed.



Use the complete plugin, not a partial hook

The online guide contains the full implementation and retry handling. This is a custom integration, not an official MyTrackr WordPress plugin.

07 / PAYSTACK

Find the right keys. Verify before recording.

Paystack processes the payment. MyTrackr records the verified financial event. Each service uses its own credentials.

MyTrackr secret

`mt_sk_…`

Authenticates your backend's event requests to MyTrackr.

Paystack public

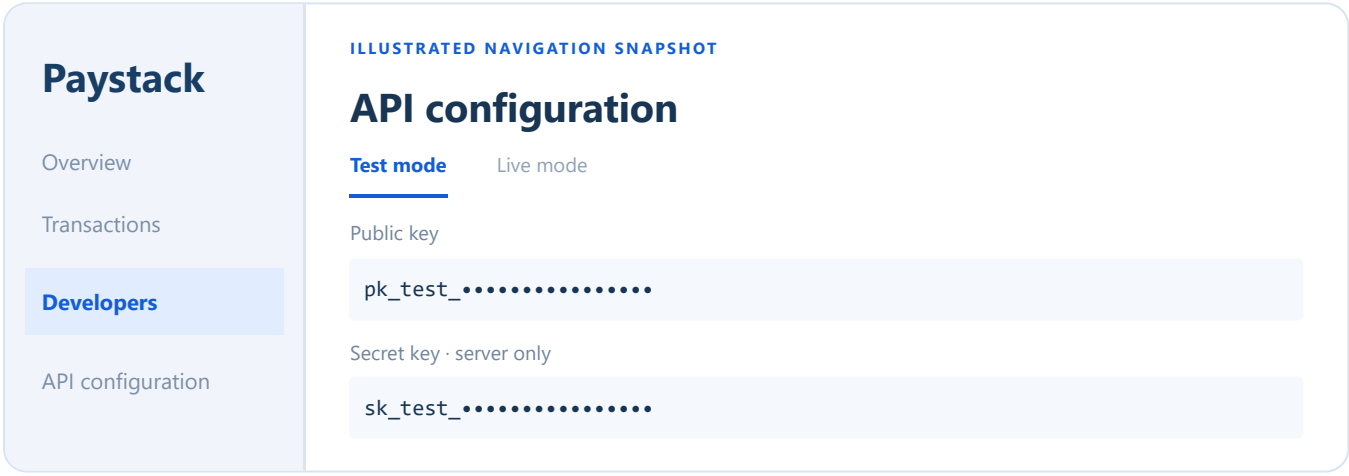
`pk_test_… / pk_live_…`

Used by the payment interface. Match the intended environment.

Paystack secret

`sk_test_… / sk_live_…`

Used only by your backend to verify payment.



Navigation aid, not a signed-in Paystack screenshot. Look under Developers; the classic dashboard uses Settings → API Keys & Webhooks. Labels can vary.

1 **Locate your keys**

Sign in to Paystack Dashboard. Copy the required keys with the account permissions and password prompts provided.

2 **Verify on your backend**

Check the transaction reference with Paystack. Confirm success and match your stored order's amount, currency, merchant, and reference.

3 **Record the verified order**

Mark the order paid, use the verified payment date, convert kobo to naira once, then call your MyTrackr sender.

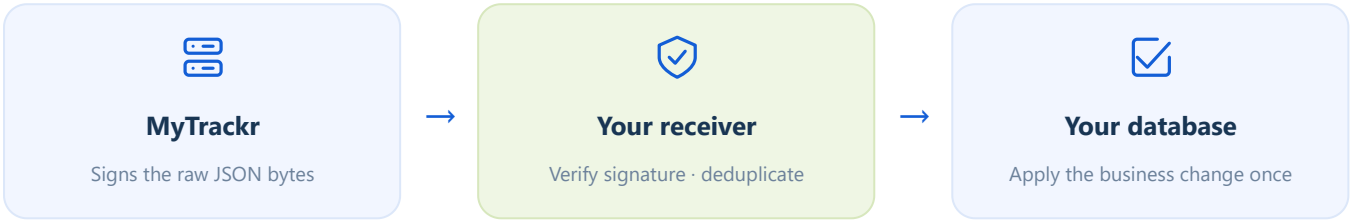
Avoid counting the sale twice

If the same transactions are already imported through MyTrackr's direct Paystack connection, reconcile that path before also sending website events.

08 / WEBHOOKS

Verify the signature. Process each delivery once.

Events travel from your server to MyTrackr. Optional webhooks travel from MyTrackr back to your server.



1 Build a receiving endpoint

Use a public HTTPS URL on port 443. Private addresses and redirects are rejected. Save it in Integrations → Settings.

2 Store the signing secret

Use your integration's `mt_whsec_…` value. Rotate it under Webhooks if lost, then update the receiver immediately.

NODE.JS · SIGNATURE VERIFICATION HELPER

```
import { createHmac, timingSafeEqual } from 'node:crypto';
function verify(rawBody, signature, secret) {
  if (!secret || typeof signature !== 'string') return false;
  const expected = 'sha256=' + createHmac('sha256', secret)
    .update(rawBody).digest('hex');
  const received = Buffer.from(signature);
  const correct = Buffer.from(expected);
  return received.length === correct.length
    && timingSafeEqual(received, correct);
}
// Pass the raw bytes before parsing JSON.
```

Deduplicate durably

Use `x-mytrackr-delivery-id` with a unique constraint. Apply your business change and mark the delivery processed atomically.

Acknowledge safely

Return a successful response for an already processed delivery. A valid signature proves origin; it does not prevent a replay.

Retry from the dashboard

Failed deliveries with fewer than five retries can be retried for enabled integrations, using the current endpoint and signing secret. The dashboard has no automatic retry scheduler.

09 / LAUNCH CHECKLIST

Make the first real order a confident one.

Check credentials, money, and recovery before you connect your production business.

Before launch

- ✓ Active website-linking plan and verified account.
- ✓ Exact HTTPS origins and a public HTTPS receiver if using webhooks.
- ✓ Separate server variables for MyTrackr, Paystack, and signing secrets.
- ✓ Amounts in major units, with the verified currency and payment date.
- ✓ Stable external IDs; duplicate delivery and timeout recovery checked.
- ✓ A single recording path, or a defined reconciliation strategy.

Test in a separate business.

MyTrackr has no isolated integration sandbox. Sample events affect that business's financial records.

Paystack test keys avoid live payment charges, but they do not turn MyTrackr events into sandbox records.

When a request fails

Response	What to do
400	Check event fields, timestamps, numeric amounts, and origin settings.
401	Check the integration API-key header is present.
403	Check the key, revoked status, owner account, and subscription expiry.
429	Pause, honour Retry-After when present, and use bounded backoff.
Timeout / 5xx	Retry with the same external ID; the first request may have succeeded.

Monitor accepted events

Compare the first real order's reference with Dashboard → Events and review webhook status. The dashboard does not log every rejected HTTP request.

Your next step

API keys ↗ | Backend ↗ | React ↗ | Vue ↗ | Flutter ↗ | WordPress ↗ | Paystack ↗ | Webhooks ↗ | Launch checklist ↗

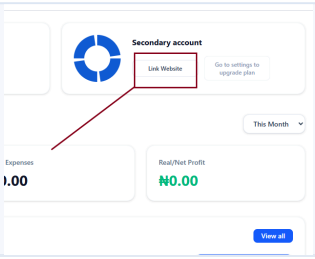
MAIN DASHBOARD / WEBSITE SETUP

Two ways to connect.

Create your website integration in the main MyTrackr dashboard or the developer dashboard. Both use the same integration service and account.

1 Open Link Website

Sign in at dashboard.mytrackrservices.com/dashboard. On the Secondary account card, select **Link Website**.

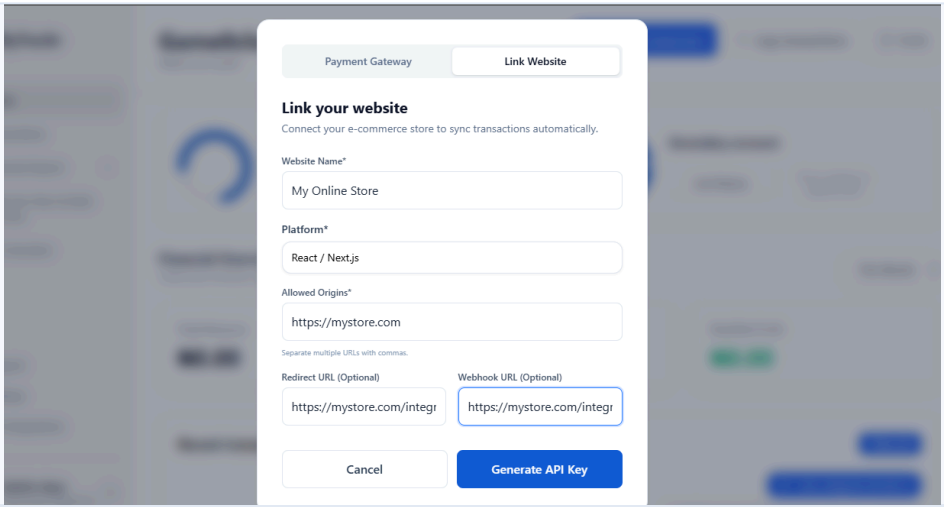


Find Link Website on the Secondary account card

Main MyTrackr dashboard walkthrough · example values

2 Complete the website form

Choose **Link Website**. Enter My Online Store, React / Next.js, and https://mystore.com. Separate origins with commas. Optional redirect and webhook URLs must be endpoints you implement; leave them blank until ready.



Link Website · example website settings

Main MyTrackr dashboard walkthrough · example values

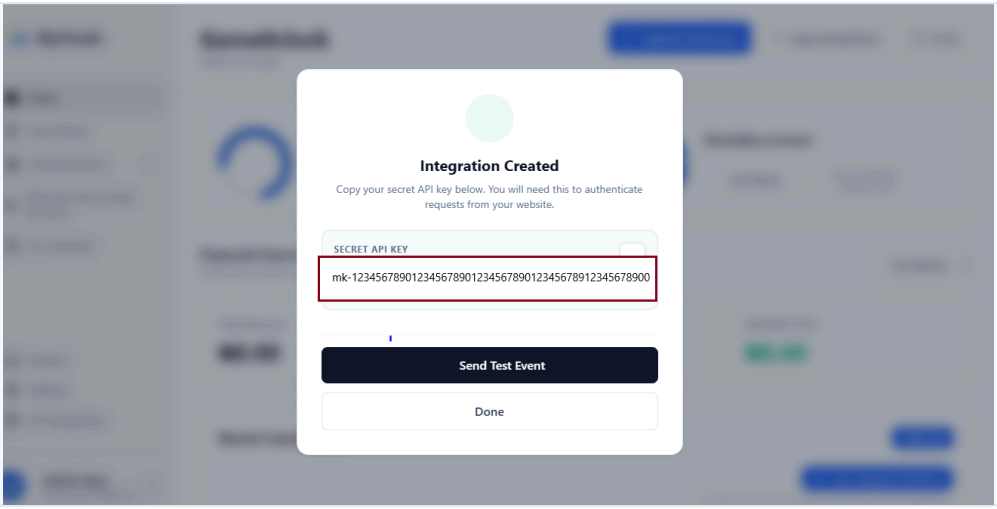
Use either dashboard

The same account can manage this integration in the developer dashboard. Do not create it twice. Your active subscription must include website linking.

MAIN DASHBOARD / CREDENTIALS

Save your key. Connect your server.

Select Generate API Key, save the returned secret in your server secret settings, then select Done.



Integration Created · pictured key is a placeholder, not a usable credential

Main MyTrackr dashboard walkthrough · example values

BASH / CURL · EQUIVALENT WEBSITE CREATION REQUEST

```
curl -X POST "https://api.mytrackrservices.com/integrations" \
  -H "Authorization: Bearer $MYTRACKR_ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  --data '{
    "name": "My Online Store",
    "platform": "react",
    "allowedOrigins": ["https://mystore.com"],
    "redirectUrl": "https://mystore.com/integrations/mytrackr/callback",
    "webhookUrl": "https://mystore.com/integrations/mytrackr/webhook"
  }'
```

Authenticate as the account owner

Set MYTRACKR_ACCESS_TOKEN to your account access token. An app session cookie also works. The older planSlug field is accepted but ignored; your actual subscription controls access.

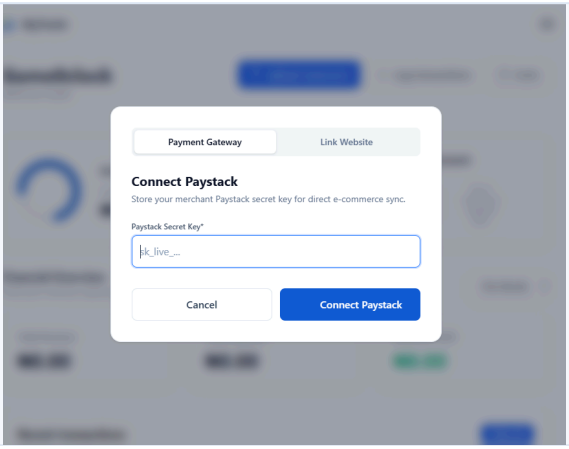
Finish the connection

Creation alone does not send website events. Follow the backend guide. Save apiKey and webhookSigningSecret from the creation response; if your main UI omits the signing secret, rotate it under Webhooks in the developer dashboard. Send Test Event affects records; there is no isolated sandbox.

MAIN DASHBOARD / PAYSTACK

Connect your merchant Paystack.

Open Link Website from the Secondary account card, then choose the Payment Gateway tab.



Payment Gateway · paste your merchant Paystack secret key

Main MyTrackr dashboard walkthrough · example values

1 Connect Paystack

Paste the Paystack Secret Key and select **Connect Paystack**. MyTrackr stores it encrypted and never returns the full value. Your account needs Paystack linking access.

BASH / CURL · EQUIVALENT MERCHANT CONNECTION

```
curl -X POST "https://api.mytrackrservices.com/integrations/paystack/connect" \
  -H "Authorization: Bearer $MYTRACKR_ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  --data "{\"secretKey\":\"$PAYSTACK_SECRET_KEY\"}"
```

BASH / CURL · REQUEST SYNC AFTER CONNECTING

```
curl -X POST "https://api.mytrackrservices.com/integrations/paystack/sync" \
  -H "Authorization: Bearer $MYTRACKR_ACCESS_TOKEN" \
  -H "Content-Type: application/json" --data '{}'
```

Where each option is available

Website integrations can be created in either dashboard. The merchant Paystack connection form is currently in the main dashboard. Connecting the key and requesting a sync are separate operations.

Keep the credentials separate

Set PAYSTACK_SECRET_KEY to your merchant secret, not your MyTrackr API key. Reconcile direct imports before sending the same sales as website events to avoid counting them twice.